

PANDAS

COMPLETE MASTERY HANDBOOK

PART ONE

Series · DataFrames · Cleaning · Merging · GroupBy · Pivoting

	Name	ID	CGPA	Dept
0	Adar	39	3.30	ICT
1	Jahid	25	4.00	CSE
2	Dipto	34	3.70	EEE

```
pd.DataFrame(student_dictionary_table)
```

Compiled & Structured for Exam-Ready Mastery

Adar Dey · Algorithm Learner and AI Enthusiastic

Table of Contents

Part 01 — Introduction to Series & DataFrames

- 1.1 Why Pandas?
- 1.2 Pandas Series — Initialization & Custom Indexing
- 1.3 DataFrame Construction Blueprints
- 1.4 Column Extraction, Selection & Mutation Pipelines
- 1.5 Location Indexing Metrics — .loc vs .iloc
- 1.6 Multi-Criteria Conditional Filtering Pipelines

Part 02 — Advanced Missing Data Cleaning Engines

- 2.1 Missing Value Auditing & Detection Schemas
- 2.2 Row Elimination with Dynamic Thresholds (.dropna)
- 2.3 Static & Target-Mapped Replacement Pipelines (.fillna)
- 2.4 Automated Statistical Mean Imputation

Part 03 — Data Merging, Concatenation & Joins

- 3.1 SQL-Style Relational Database Merging (pd.merge)
- 3.2 Axis-Based Structural Concatenation (pd.concat)
- 3.3 Index-Based Matrix Merging (DataFrame.join)

Part 04 — Data Aggregations & GroupBy Mechanics

- 4.1 GroupBy Iteration — Deconstructing the Split Stage
- 4.2 Single-Column Categorical Aggregations
- 4.3 Multi-Level Hierarchical Grouping Pipelines
- 4.4 Descriptive Matrix Profiling Operations (.agg)

Part 05 — Pivot Tables & Crosstab Engines

- 5.1 DateTime Metadata Extraction via the .dt Accessor
- 5.2 The Reshaping Engine — Pandas Pivot Tables (pd.pivot_table)
- 5.3 Categorical Frequency Analyses (pd.crosstab)

Part 06 — Metadata Auditing & Custom Function Mapping

- 6.1 Structural Meta Auditing (.shape, .info, .describe)
- 6.2 In-Place Vectorized Element Broadcasting
- 6.3 Custom Element Mapping Operations (.apply via Lambdas)

PART 01

Introduction to Series & DataFrames

Core Structures, Construction & Selection Mechanics

6 Core Topics · Fully Worked Examples & Outputs

PART 1 • TOPIC 1

Why Pandas?

Pandas stands for **Panel Data** — the premier Python Data Analysis Library, built directly on top of NumPy.

Key advantages:

- ▶ The premier data manipulation and analysis tool built on top of NumPy.
- ▶ Introduces two key structural objects: Series (1D) and DataFrames (2D tables).
- ▶ Seamlessly handles heterogeneous data types across columns.
- ▶ Built-in label-based indexing, alignment, dropping, and boolean slicing.

>_ PYTHON

```
import numpy as np
import pandas as pd
```

◆ KEY INSIGHT

Pandas is to tabular data what NumPy is to numerical arrays — almost every real-world data pipeline in Python passes through it.

PART 1 • TOPIC 2

Pandas Series — Initialization & Custom Indexing

A **Series** is a one-dimensional labeled array capable of holding any data type. It consists of an axis labels array (called the index) and the actual data values.

>_ PYTHON

```
# A. From a raw NumPy array (default numeric indexing)
numpy_raw_data = np.array([2, 4, 1, 6])
default_series = pd.Series(numpy_raw_data)

# B. With custom explicit string indices
custom_string_indices = np.array(['a', 'b', 'c', 'd'])
labeled_series = pd.Series(numpy_raw_data, index=custom_string_indices)

# C. Direct from a standard Python dictionary
dictionary_source = {'a': 11, 'b': 12, 'c': 13}
dictionary_series = pd.Series(dictionary_source)
```

→ OUTPUT

Default Indexed Series:	Custom Labeled Series:	From Dictionary:
0 2	a 2	a 11
1 4	b 4	b 12
2 1	c 1	c 13
3 6	d 6	dtype: int64

```
dtype: int64      dtype: int64
```

PART 1 • TOPIC 3

DataFrame Construction Blueprints

A **DataFrame** is a two-dimensional tabular data structure with rows and columns — think of it as an optimized, programmatic SQL table or Excel spreadsheet.

> _ PYTHON

```
# A. From a dictionary of aligned lists
student_dictionary_table = {
    "Name": ["Adar", "Jahid", "Dipto"],
    "ID": [39, 25, 34],
    "CGPA": [3.30, 4.00, 3.70]
}
df_from_dict = pd.DataFrame(student_dictionary_table)

# B. From a nested list with explicit column headers
student_nested_list = [
    ["Adar", 39, 3.30],
    ["Jahid", 25, 4.00],
    ["Dipto", 34, 3.70]
]
column_headers = ["Name", "ID", "CGPA"]
df_student_profile = pd.DataFrame(student_nested_list, columns=column_headers)
```

→ OUTPUT

```
   Name  ID  CGPA
0  Adar  39   3.3
1  Jahid 25   4.0
2  Dipto 34   3.7
```

◆ KEY INSIGHT

Both construction methods yield an identical resulting table — choose whichever source format your raw data naturally arrives in.

PART 1 • TOPIC 4

Column Extraction, Selection & Mutation Pipelines

Columns can be isolated with single or multi-column selection lists, dynamically appended, and permanently or temporarily dropped using the `axis` parameter — `axis=1` targets columns, `axis=0` targets rows.

> _ PYTHON

A. Single & multi-column selection

```
single_column_selection = df_student_profile["Name"]
multi_column_selection = df_student_profile[["Name", "CGPA"]]
```

B. Dynamically appending/mutating a new column

```
df_student_profile["Age"] = [23, 24, 22]
```

C. Dropping columns/rows

```
df_temporary_drop = df_student_profile.drop('Age', axis=1)    # non-destructive copy
df_student_profile.drop('Age', axis=1, inplace=True)         # permanent overwrite
df_multi_dropped = df_student_profile.drop(['ID', 'CGPA'], axis=1) # batch column drop
df_row_dropped = df_student_profile.drop(0, axis=0)           # row elimination
```

→ OUTPUT

After inplace drop of 'Age':	After batch drop ['ID','CGPA']:	After row 0 dropped:
<pre>Name ID CGPA 0 Adar 39 3.3 1 Jahid 25 4.0 2 Dipto 34 3.7</pre>	<pre>Name 0 Adar 1 Jahid 2 Dipto</pre>	<pre>Name ID CGPA 1 Jahid 25 4.0 2 Dipto 34 3.7</pre>

◆ KEY INSIGHT

Without `inplace=True`, `.drop()` always returns a fresh copy — the original DataFrame is left completely untouched.

PART 1 • TOPIC 5

Location Indexing Metrics — `.loc` vs `.iloc`

`.loc`: Label-Based index selection — accesses data points using literal named indices/labels.

`.iloc`: Integer-Position Based selection — accesses data points via zero-indexed integer slots.

> _ PYTHON

```
df_student_profile.loc[0]          # single row, label-based
df_student_profile.loc[[0, 1]]    # batch multi-row label selection
df_student_profile.iloc[2]        # absolute integer position row

# Dual-axis intersection — quadrant slicing
# Syntax: dataframe.loc[row_labels][column_labels]
sub_grid_intersection = df_student_profile.loc[[1, 2]][["ID", "CGPA"]]
```

→ OUTPUT

```
Quadrant Intersection Result:
  ID CGPA
1  25  4.0
2  34  3.7
```

PART 1 • TOPIC 6

Multi-Criteria Conditional Filtering Pipelines

Combining multiple boolean masks with the vectorized `&` operator lets you filter rows that satisfy several conditions simultaneously — the pandas equivalent of a SQL `WHERE` clause with `AND` logic.

>_ PYTHON

```
master_table_source = [
    ["Adar", 39, 3.30, 23],
    ["Jahid", 25, 4.00, 24],
    ["Dipto", 34, 3.70, 22]
]

df_master_profile = pd.DataFrame(master_table_source, columns=["Name", "ID", "CGPA", "Age"])

conditional_mask_filter = (df_master_profile["Age"] > 23) & (df_master_profile["CGPA"] > 3.5)

filtered_data_pipeline = df_master_profile[conditional_mask_filter]
```

→ OUTPUT

	Name	ID	CGPA	Age
1	Jahid	25	4.0	24

◆ KEY INSIGHT

Always wrap each individual comparison in parentheses when chaining conditions with `&` or `|` — Python's operator precedence requires it.

PART 02

Advanced Missing Data Cleaning Engines

Detection, Elimination, Filling & Statistical Imputation

4 Core Topics · Fully Worked Examples & Outputs

PART 2 • TOPIC 1

Missing Value Auditing & Detection Schemas

Real-world data almost always contains gaps. Pandas provides a layered toolkit to detect exactly where those gaps live before deciding how to handle them.

>_ PYTHON

```
unclean_data_source = {
    "A": [1, 2, 3, 4, 5],
    "B": [1, 2, 3, 4, 5],
    "C": [1, 2, 3, np.nan, np.nan],
    "D": [1, np.nan, np.nan, np.nan, 5]
}
df_dirty = pd.DataFrame(unclean_data_source)

# A. Element-wise boolean null mask
boolean_null_mask = df_dirty.isna()

# B. Aggregated vertical frequency counts per column
column_null_counts = df_dirty.isna().sum()

# C. Structural existence flags — does ANY null exist per column?
column_null_presence = df_dirty.isna().any()
```

→ OUTPUT

Boolean_null_mask:					Missing Values Count Per Column:	Global Null Presence Flags:		
A	B	C	D	E				
0	False	False	False	False	A	0	A	False
1	False	False	False	True	B	0	B	False
2	False	False	False	True	C	2	C	True
3	False	False	False	True	D	3	D	True
4	False	False	False	True	dtype: int64		dtype: bool	

PART 2 • TOPIC 2

Row Elimination with Dynamic Thresholds (.dropna)

thresh=x sets a strict survival condition: a row **MUST** contain at least x non-NaN elements to be retained — giving you fine control beyond the default 'drop if ANY value is missing' behavior.

>_ PYTHON

```
df_aggressive_drop = df_dirty.dropna()      # drop if ANY element is NaN

df_lenient_drop = df_dirty.dropna(thresh=1)  # needs only 1 valid cell to survive

df_over_strict_drop = df_dirty.dropna(thresh=5)  # impossible — every row dropped
```

→ OUTPUT

Aggressive drop (only pristine rows survive):

```
A B C D
0 1 1 1.0 1.0
```

Dropped Rows Using Threshold = 1 (Lenient):

```
A B C D
0 1 1 1.0 1.0
1 2 2 2.0 NaN
2 3 3 3.0 NaN
3 4 4 NaN NaN
4 5 5 NaN 5.0
```

Threshold = 5 (unattainable criteria):

Empty DataFrame

Columns: [A, B, C, D]

Index: []

PART 2 • TOPIC 3

Static & Target-Mapped Replacement Pipelines (.fillna)

Missing values can be replaced globally with one static fallback, or precisely mapped per column using a dictionary — giving you column-aware control over how each feature's gaps get patched.

>_ PYTHON

A. Global uniform value replacement

```
df_blank_filled = df_dirty.fillna("NANA")
```

B. Precise per-column target mapping

```
column_replacement_map = {'A': 0, 'B': 100, 'C': 300, 'D': 400}
```

```
df_mapped_fill = df_dirty.fillna(value=column_replacement_map)
```

→ OUTPUT

Globally substituted static strings Layout:

```
A B C D
0 1 1 1.0 1.0
1 2 2 2.0 NANA
2 3 3 3.0 NANA
3 4 4 NANA NANA
4 5 5 NANA 5.0
```

Precisely Mapped Imputation Output:

```
A B C D
0 1 1 1.0 1.0
1 2 2 2.0 400.0
2 3 3 3.0 400.0
3 4 4 300.0 400.0
4 5 5 300.0 5.0
```

PART 2 • TOPIC 4

Automated Statistical Mean Imputation

Imputing missing values with each column's own statistical mean prevents loss of data volume while keeping downstream statistical distributions stable — a far gentler approach than dropping rows outright.

>_ PYTHON

```
# Dynamically calculates mathematical means for each numerical column
# and injects them directly back into that column's missing slots
df_imputed_mean = df_dirty.fillna(df_dirty.mean())
```

→ OUTPUT

```
   A  B  C    D
0  1  1  1.0  1.000000
1  2  2  2.0  2.333333 ← Column D mean override
2  3  3  3.0  2.333333 ← Column D mean override
3  4  4  2.0  2.333333 ← Column C & D mean override
4  5  5  2.0  5.0
```

◆ KEY INSIGHT

`df.mean()` computes a per-column mean automatically, so `fillna()` matches each column to its own average — never a single global average.

PART 03

Data Merging, Concatenation & Joins

Relational Merges, Axis Stacking & Index-Based Joins

3 Core Topics · Fully Worked Examples & Outputs

PART 3 • TOPIC 1

SQL-Style Relational Database Merging (pd.merge)

pd.merge() combines two DataFrames on shared key columns, exactly like a SQL JOIN. The how parameter controls exactly which rows survive the merge.

how='inner' / default	how='outer'
▸ Keeps rows ONLY where the key exists in BOTH DataFrames. ▸ Default behavior when 'how' is omitted entirely. ▸ Result is always the smallest possible row count.	▸ Unions all keys from both DataFrames. ▸ Fills every non-matching cell with NaN automatically. ▸ Result retains every record from both source tables.

> _ PYTHON

```
df_employees = pd.DataFrame({
    'emp-id': [1, 2, 3, 4, 5],
    'name': ['John','Anna','Peter','Linda','Bob'],
    'department': ['HR','IT','Finance','IT','HR']
})
df_salaries = pd.DataFrame({
    'emp-id': [1, 2, 3, 16, 17],
    'salary': [60000,80000,65000,70000,90000],
    'bonus': [5000,10000,7000,8000,12000]
})

pd.merge(df_employees, df_salaries)                # implicit inner
pd.merge(df_employees, df_salaries, on="emp-id", how='inner') # explicit inner (Same output like implicit inner)
pd.merge(df_employees, df_salaries, on="emp-id", how='outer') # full outer
pd.merge(df_employees, df_salaries, on="emp-id", how='left')  # left directional
pd.merge(df_employees, df_salaries, on="emp-id", how='right') # right directional
```

→ OUTPUT

Implicit Inner(Default) / Explicit Inner Merge:

	emp-id	name	department	salary	bonus
0	1	John	HR	60000	5000
1	2	Anna	IT	80000	10000
2	3	Peter	Finance	65000	7000

Full Outer Union Merge:

	emp-id	name	department	salary	bonus
0	1	John	HR	60000.0	5000.0
1	2	Anna	IT	80000.0	10000.0
2	3	Peter	Finance	65000.0	7000.0
3	4	Linda	IT	NaN	NaN
4	5	Bob	HR	NaN	NaN
5	16	NaN	NaN	70000.0	8000.0
6	17	NaN	NaN	90000.0	12000.0

Left Directional Join Merge:

	emp-id	name	department	salary	bonus
0	1	John	HR	60000.0	5000.0
1	2	Anna	IT	80000.0	10000.0
2	3	Peter	Finance	65000.0	7000.0
3	4	Linda	IT	NaN	NaN
4	5	Bob	HR	NaN	NaN

Right Directional Join Merge:

	emp-id	name	department	salary	bonus
0	1	John	HR	60000	5000
1	2	Anna	IT	80000	10000
2	3	Peter	Finance	65000	7000
3	16	NaN	NaN	70000	8000
4	17	NaN	NaN	90000	12000

◆ KEY INSIGHT

Left keeps every Left row (fills unmatched Right cells with NaN); Right keeps every Right row (fills unmatched Left cells with NaN).

PART 3 • TOPIC 2

Axis-Based Structural Concatenation (pd.concat)

pd.concat() joins data assets along a physical grid axis: axis=0 stacks tables vertically (rows), while axis=1 aligns tables horizontally side-by-side.

> PYTHON

```
df_comp1 = pd.DataFrame({'A': ['a0', 'a1', 'a3'], 'B': ['b0', 'b1', 'b3'], 'C': ['c0', 'c1', 'c3']})
df_comp2 = pd.DataFrame({'A': ['a4', 'a5', 'a6'], 'B': ['b4', 'b5', 'b6'], 'C': ['c4', 'c5', 'c6']})

df_v_concat = pd.concat([df_comp1, df_comp2])          # vertical row stacking (axis=0)
df_h_concat = pd.concat([df_comp1, df_comp2], axis=1) # horizontal column alignment
```

→ OUTPUT

Vertical Concat (axis=1):

```
  A B C
0 a0 b0 c0
1 a1 b1 c1
2 a3 b3 c3
0 a4 b4 c4
1 a5 b5 c5
2 a6 b6 c6
```

Horizontal Concat (axis=1):

```
  A B C A B C
0 a0 b0 c0 a4 b4 c4
1 a1 b1 c1 a5 b5 c5
2 a3 b3 c3 a6 b6 c6
```

PART 3 • TOPIC 3

Index-Based Matrix Merging (DataFrame.join)

.join() links two DataFrames primarily on their Index keys rather than named columns — cleaner and faster when combining datasets that are already aligned by a custom index.

> PYTHON

```
df_indexed_x = pd.DataFrame({"Name": ['Alice', 'Bob', 'Charlie']}, index=[1, 2, 3])
df_indexed_y = pd.DataFrame({"score": [85, 90, 75]}, index=[2, 3, 4])

df_join_outer = df_indexed_x.join(df_indexed_y, how='outer') # unions all indices
df_join_inner = df_indexed_x.join(df_indexed_y, how='inner') # keeps matching only
```

→ OUTPUT					
Outer Join:			Inner Join:		
	Name	score		Name	score
1	Alice	NaN	2	Bob	85
2	Bob	85.0	3	Charlie	90
3	Charlie	90.0			
4	NaN	75.0			

PART 04

Data Aggregations & GroupBy Mechanics

Split-Apply-Combine, Hierarchical Grouping & Bundled Stats

4 Core Topics · Fully Worked Examples & Outputs

PART 4 • TOPIC 1

GroupBy Iteration — Deconstructing the Split Stage

When executing `.groupby()`, Pandas partitions data rows behind the scenes based on matching keys (the Split phase) before any math calculations run. Iterating the group object reveals each unique key and its matching sub-DataFrame.

> _ PYTHON

```
transaction_records = {
    'Category': ['A','B','A','B','A','B','A','B'],
    'Store': ['S1','S1','S2','S2','S1','S2','S2','S1'],
    'Sales': [100,200,150,250,120,180,200,300],
    'Quantity': [10,15,12,18,8,20,15,25],
    'Date': pd.date_range('2023-01-01', periods=8)
}
df_sales_ledger = pd.DataFrame(transaction_records)

grouped_by_category = df_sales_ledger.groupby('Category')

for group_key, sub_dataframe in grouped_by_category:
    print(f"Active Extracted Group Key: [{group_key}]")
    print(sub_dataframe)
```

→ OUTPUT

--- Base Transaction Ledger Profiles ---

	Category	Store	Sales	Quantity	Date
0	A	S1	100	10	2023-01-01
1	B	S1	200	15	2023-01-02
2	A	S2	150	12	2023-01-03
3	B	S2	250	18	2023-01-04
4	A	S1	120	8	2023-01-05
5	B	S2	180	20	2023-01-06
6	A	S2	200	15	2023-01-07
7	B	S1	300	25	2023-01-08

Active Extracted Group Key: [A]

	Category	Store	Sales	Quantity
0	A	S1	100	10
2	A	S2	150	12
4	A	S1	120	8
6	A	S2	200	15

Active Extracted Group Key: [B]

	Category	Store	Sales	Quantity
1	B	S1	200	15
3	B	S2	250	18
5	B	S2	180	20
7	B	S1	300	25

PART 4 • TOPIC 2

Single-Column Categorical Aggregations

Syntax model: `dataframe.groupby('Key')['Value'].math_function()` — this extracts a specific segment and bundles the summary into a single Series vector.

>_ PYTHON

```
category_sales_totals = df_sales_ledger.groupby('Category')['Sales'].sum()
```

→ OUTPUT

```
Category
A    570
B    930
Name: Sales, dtype: int64
```

PART 4 • TOPIC 3

Multi-Level Hierarchical Grouping Pipelines

Passing a list of distinct columns into the grouping engine generates a multi-layered MultiIndex row layout that tracks every combination across both grouping variables simultaneously.

>_ PYTHON

```
multi_level_sales_totals = df_sales_ledger.groupby(['Category', 'Store'])['Sales'].sum()
```

→ OUTPUT

```
   Category Store
A      S1     220
      S2     350
B      S1     500
      S2     430
Name: Sales, dtype: int64
```

PART 4 • TOPIC 4

Descriptive Matrix Profiling Operations (.agg)

Passing a list of statistical function names directly into .agg() computes multiple metrics at once, building a comprehensive summary panel from a single raw data column.

>_ PYTHON

```
flat_sales_average = df_sales_ledger['Sales'].mean() # 187.5

sales_comprehensive_profile = df_sales_ledger['Sales'].agg(['sum', 'mean', 'min', 'max', 'count', 'std', 'median'])
```

→ OUTPUT

sum

1500.000000

mean

187.500000

min

100.000000

max

300.000000

count

8.000000

std

66.062741

median

190.000000

Name: Sales, dtype: float64

PART 05

Pivot Tables & Crosstab Engines

DateTime Extraction, Matrix Reshaping & Frequency Auditing

3 Core Topics · Fully Worked Examples & Outputs

PART 5 • TOPIC 1

DateTime Metadata Extraction via the .dt Accessor

The `.dt` accessor unlocks calendar-aware metadata straight from a timestamp Series — extracting names, quarters, and other temporal features without any manual date-math.

> PYTHON

```
raw_ledger_data = {
    'Date': pd.date_range('2023-01-01', periods=20),
    'Product': ['A','B','C','D'] * 5,
    'Region': ['East','West','North','South'] * 5,
    'Sales': np.random.randint(100, 1000, 20),
    'Units': np.random.randint(10, 100, 20),
    'Rep': ['John','Mary','Bob','Alice'] * 5
}
df_source = pd.DataFrame(raw_ledger_data)

df_source['Month'] = df_source['Date'].dt.month_name()
df_source['Quarter'] = 'Q' + df_source['Date'].dt.quarter.astype(str)
```

PART 5 • TOPIC 2

The Reshaping Engine — Pandas Pivot Tables (`pd.pivot_table`)

Four critical parameters control every pivot: **index** sets the row labels, **columns** sets the column headings, **values** selects the numeric column being aggregated, and **aggfunc** is the mathematical engine that compresses matching groups (defaults to 'mean').

The column assigned to 'values' must strictly contain numeric data — passing text columns into a mathematical aggfunc throws an explicit exception.

> PYTHON

```
# A. Single custom aggregation metric
pivot_median_sales = pd.pivot_table(
    data=df_source, values="Sales", index="Region", columns="Product", aggfunc="median"
)

# B. Multi-value multi-layered dimension reshaping
pivot_multi_metric = pd.pivot_table(
    data=df_source, values=['Sales', 'Units'], index='Region',
    columns='Product', aggfunc='mean'
)
```

→ OUTPUT

Pivot Table: Median Sales across Product x Region Matrix:

Product	A	B	C	D
Region				
East	537.0	NaN	NaN	NaN
North	NaN	NaN	544.0	NaN
South	NaN	NaN	NaN	562.0
West	NaN	548.0	NaN	NaN

Hierarchical Pivot Table: Mean Metrics Matrix Across Product-Region Axes:

Product	Sales			Units				
	A	B	C	D	A	B	C	D
Region								
East	350.2	NaN	NaN	NaN	54.8	NaN	NaN	NaN
North	NaN	NaN	655.2	NaN	NaN	NaN	46.2	NaN
South	NaN	NaN	NaN	301.2	NaN	NaN	NaN	80.4
West	NaN	491.2	NaN	NaN	NaN	44.2	NaN	NaN

◆ KEY INSIGHT

NaN values represent empty slots where no transactional row existed for that specific row/column intersection — this is expected, not an error.

PART 5 • TOPIC 3

Categorical Frequency Analyses (pd.crosstab)

pd.crosstab() computes a frequency contingency map that displays raw element co-occurrence counts across distinct structural groupings — perfect for auditing category distributions.

>_ PYTHON

contingency_frequency_matrix = pd.crosstab(
 index=df_source['Region'],
 columns=df_source['Product']
)

→ OUTPUT

Product	A	B	C	D
Region				
East	5	0	0	0
North	0	0	5	0
South	0	0	0	5
West	0	5	0	0

PART 06

Metadata Auditing & Custom Function Mapping

Structural Audits, Broadcasting & Lambda-Based Transformations

3 Core Topics · Fully Worked Examples & Outputs

PART 6 • TOPIC 1

Structural Meta Auditing (.shape, .info, .describe)

.shape returns a strict (rows, columns) tuple. .info() prints index ranges, non-null counts, dtypes, and memory footprint. .describe() computes averages, standard deviations, and quartile cutoffs for every numeric column.

> _ PYTHON

```
df_metrics = pd.DataFrame({
    'A': [1,2,3,4,5], 'B': [10,20,30,40,50], 'C': [100,200,300,400,500]
})

df_metrics.shape      # (5, 3)
df_metrics.info()     # index range, dtypes, memory usage
df_metrics.describe() # count, mean, std, min, 25/50/75%, max
```

→ OUTPUT

DataFrame Shape Demarcation Boundaries: (5, 3)

System Architecture Diagnostic Summary:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 5 entries, 0 to 4

Data columns (total 3 columns):

Column Non-Null Count Dtype

```
---  ---  ---
0  A      5 non-null   int64
1  B      5 non-null   int64
2  C      5 non-null   int64
```

dtypes: int64(3)

memory usage: 252.0 bytes

	A	B	C
count	5.000000	5.000000	5.000000
mean	3.000000	30.000000	300.000000
std	1.581139	15.811388	158.113883
min	1.000000	10.000000	100.000000
25%	2.000000	20.000000	200.000000
50%	3.000000	30.000000	300.000000
75%	4.000000	40.000000	400.000000
max	5.000000	50.000000	500.000000

PART 6 • TOPIC 2

In-Place Vectorized Element Broadcasting

Just like NumPy arrays, Pandas Series execute vectorized math natively. Adding a scalar to a Series broadcasts the operation across every row instantly — no explicit loop required.

> _ PYTHON

```
df_metrics['A'] = df_metrics['A'] + 10
```

→ OUTPUT

```
0  11
1  12
2  13
3  14
4  15
Name: A, dtype: int64
```

PART 6 • TOPIC 3

Custom Element Mapping Operations (.apply via Lambdas)

`.apply()` runs custom data transformations across rows or columns. Paired with an inline lambda, it maps a custom operation over every element in a data channel without slow, explicit Python loops.

> _ PYTHON

```
df_metrics['D'] = df_metrics["B"].apply(lambda x: x ** 2)
```

→ OUTPUT

```
   A  B  C  D
0  11 10 100 100
1  12 20 200 400
2  13 30 300 900
3  14 40 400 1600
4  15 50 500 2500
```

◆ KEY INSIGHT

Vectorized `.apply()` with a lambda is dramatically faster than an explicit Python for-loop over DataFrame rows on large datasets.